

Documento de Arquitectura y Selección de Estructuras de Datos

Proyecto: Sistema de Gestión de Transporte Público

Materia: Estructuras de Datos

1. Introducción y Enfoque Arquitectónico

El presente documento argumenta las decisiones de diseño de software y la selección de estructuras de datos implementadas para el Sistema de Gestión de Transporte Público. El objetivo principal fue construir un sistema robusto, eficiente y altamente escalable en memoria, cumpliendo con la restricción técnica absoluta de no utilizar ninguna clase o interfaz perteneciente al framework de colecciones de Java (`java.util.*`).

Para garantizar un código limpio y modular, el sistema fue diseñado bajo el **Patrón Arquitectónico MVC (Modelo-Vista-Controlador)**. Esta separación de responsabilidades asegura que las estructuras de datos personalizadas (Modelo) operen de forma completamente aislada de la interfaz gráfica (Vista), comunicándose a través de un Controlador central utilizando Arreglos Primitivos y Patrones DTO (Data Transfer Objects).

2. Argumentación de Estructuras de Datos Seleccionadas

Para resolver los requerimientos de gestión de paradas, conexiones y cálculo de rutas óptimas, se diseñó un ecosistema de estructuras de datos construidas desde cero.

2.1. Lista Enlazada Simple Genérica (`ListaSimple<T>`)

- **Uso en el sistema:** Actúa como la estructura base para el almacenamiento dinámico. Se utiliza para mantener el catálogo global de vértices (paradas) en el Grafo y para gestionar las listas de adyacencia (conexiones) de cada nodo.
- **Justificación de diseño:** A diferencia de los arreglos estáticos nativos, una lista enlazada permite un crecimiento dinámico de la memoria en tiempo de ejecución. Al crear una lista genérica, se garantizó la reutilización del código (`ListaSimple<Vertice>` y `ListaSimple<Arista>`), aplicando principios de tipado fuerte sin depender de utilidades externas.

2.2. Grafo basado en Listas de Adyacencia (`Grafo<T>`, `Vertice<T>`, `Arista<T>`)

- **Uso en el sistema:** Es el núcleo del modelo. Representa la topología de la red de transporte público, donde los Vértices son las paradas de autobús y las Aristas son las rutas o trayectos entre ellas.
- **Justificación de diseño:** Se eligió una representación por *Listas de Adyacencia* en lugar de una *Matriz de Adyacencia*. Las redes de transporte público son matemáticamente "grafos dispersos" (no todas las paradas están conectadas directamente con todas las demás). Una matriz habría desperdiciado grandes

cantidades de memoria continua ($O(V^2)$), mientras que la lista de adyacencia optimiza el consumo de RAM a $O(V + E)$ (Vértices + Aristas), permitiendo iteraciones mucho más rápidas al buscar conexiones vecinas.

2.3. Cola de Prioridad (**ColaPrioridad<T>**)

- **Uso en el sistema:** Estructura fundamental para la ejecución matemática del algoritmo de enrutamiento (Dijkstra).
- **Justificación de diseño:** Para encontrar el camino más corto, el sistema necesita evaluar siempre la parada alcanzable más cercana. Implementar una cola de prioridad basada en un criterio de "peso" (distancia/tiempo) garantiza que en cada iteración del algoritmo se extraiga el nodo con el costo acumulado más bajo, optimizando el tiempo de respuesta en redes con alta densidad de paradas.

2.4. Pila (**Pila<T>**)

- **Uso en el sistema:** Reconstrucción del historial de ruta óptima calculada.
- **Justificación de diseño:** Una vez que el algoritmo de búsqueda de rutas llega al destino, el rastro de los nodos predecesores se encuentra en orden inverso (del destino al origen). La Pila, por su naturaleza **LIFO** (Last-In, First-Out), es la estructura matemáticamente perfecta para este problema: se introducen las paradas desde el final hacia el inicio y, al desapilarlas (**pop**), se obtiene automáticamente el trayecto ordenado lógicamente para presentarlo al usuario.

3. Decisiones de Diseño y Patrones Adicionales

1. **Aislamiento del Algoritmo (Principio SRP de SOLID):** El cálculo de la ruta óptima no se dejó dentro del servicio principal. Se extrajo a una clase especializada (**DijkstraRouter**). Esta decisión de diseño facilita el mantenimiento y permite que en el futuro se puedan inyectar nuevos algoritmos (como A* o Bellman-Ford) sin modificar el Grafo ni el Controlador.
2. **Uso de DTOs (Data Transfer Objects):** Para evitar exponer las entidades internas y romper el encapsulamiento, el modelo empaqueta la información en **ParadaDTO** y **ConexionDTO**. Esto previene que la Vista altere accidentalmente la integridad de la red de transporte.
3. **Control de Excepciones:** Se diseñó un flujo de excepciones controladas (**IllegalArgumentException**, **RuntimeException**) para atrapar inconsistencias de negocio (pesos negativos, paradas duplicadas o rutas inalcanzables), evitando caídas del sistema y mejorando la confiabilidad de la aplicación.